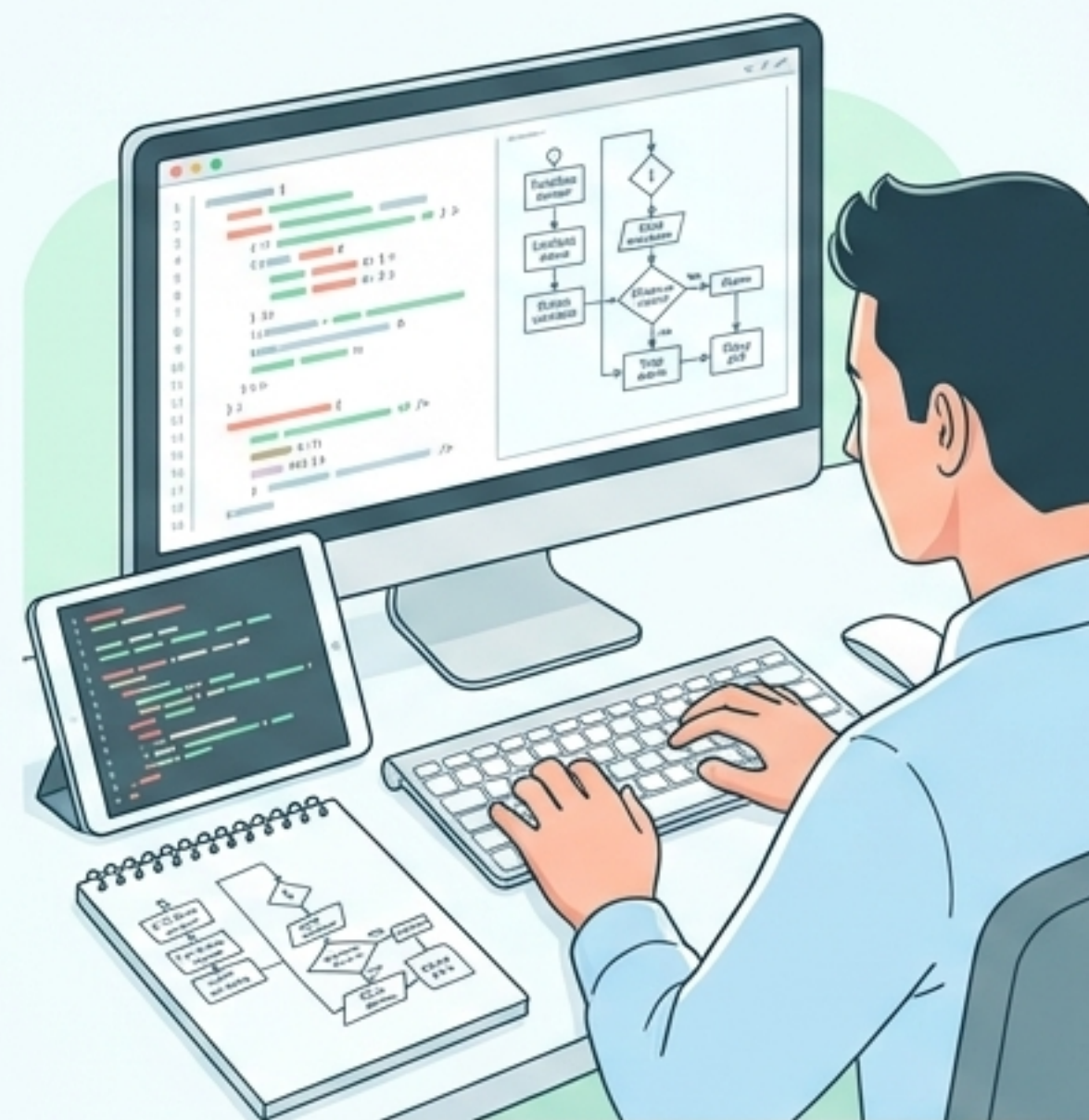


- src/
- tests/
- docs/
- config/

Fuente: Tema 16. Entornos de desarrollo

"Pruebas de Código: Garantizando la Calidad"

// Metodologías, automatización y estándares para un software fiable



> inicializar_calidad()

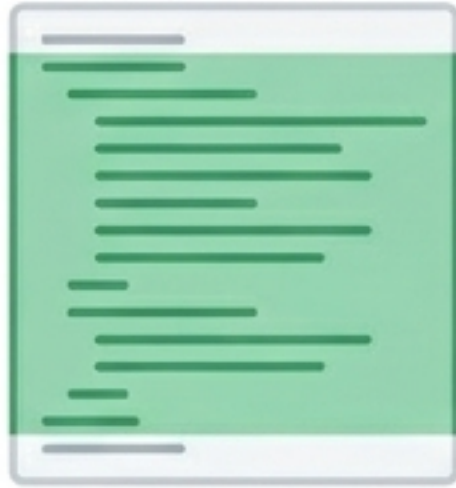


SYS_LOG

Los usuarios exigen estándares de calidad. Como desarrolladores, debemos auditar activamente el código para proteger los datos, asegurar el flujo de trabajo y retener al cliente.

Niveles de Inspección: Filtros de Ejecución

Pruebas de Cobertura



Objetivo: Ejecutar al menos una vez cada línea de código.
Muchas herramientas IDE muestran automáticamente cuántas veces se ejecuta cada instrucción durante la ejecución.

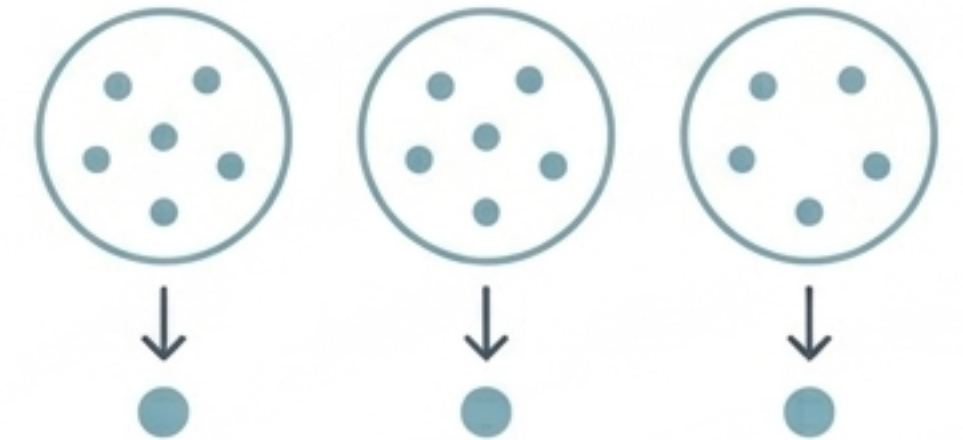
Valores Límite



Objetivo: Evaluar el comportamiento en casos excepcionales.

Ejemplo clásico: Controlar la división por cero para informar de una entrada no válida en lugar de abortar.

Clases de Equivalencia

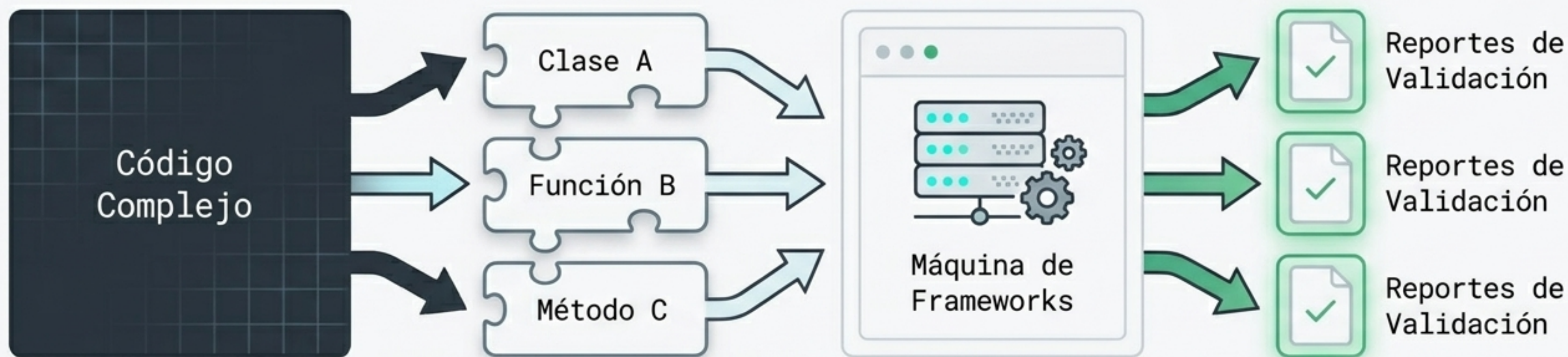


Objetivo: Reducir los casos de prueba basándose en funciones matemáticas.

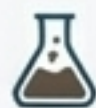
Solo se prueban valores equivalentes representativos dentro de un mismo rango lógico.

Desacoplar y Validar: El Ecosistema Unitario

El objetivo de las pruebas unitarias es aislar cada porción del software (clases, funciones) por debajo del nivel de integración para comprobar todos sus caminos posibles.



JUnit



Para Java. Integrado en IDEs (NetBeans, Eclipse). Gestiona ejecución segura y regresiones.

PHPUnit



El estándar equivalente para realizar pruebas unitarias robustas en entornos PHP.

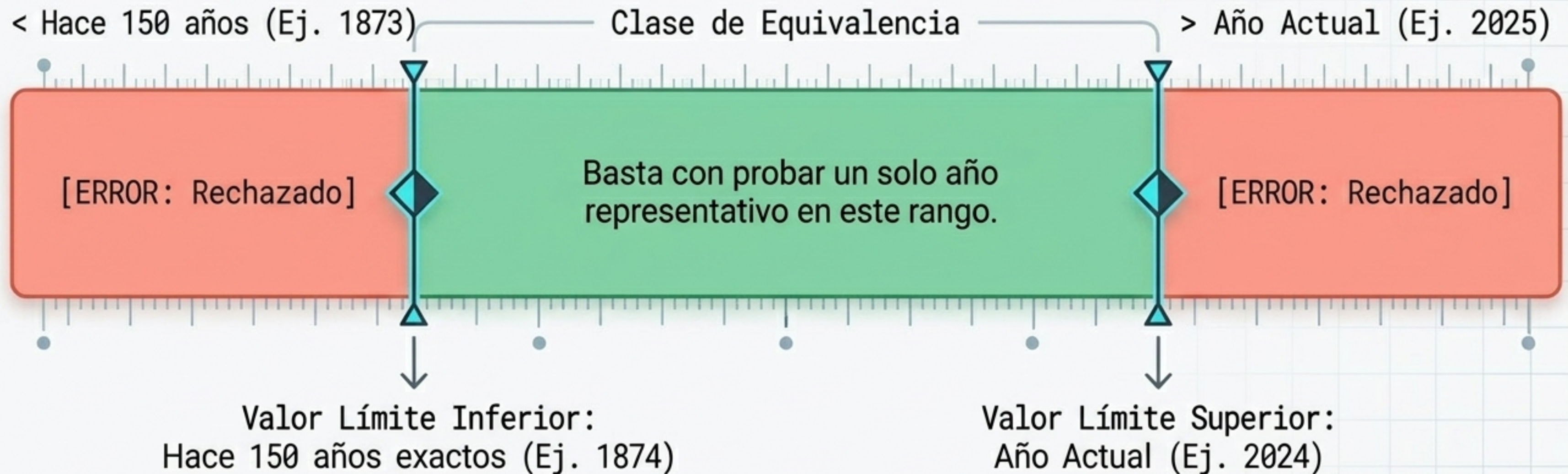
NUnit



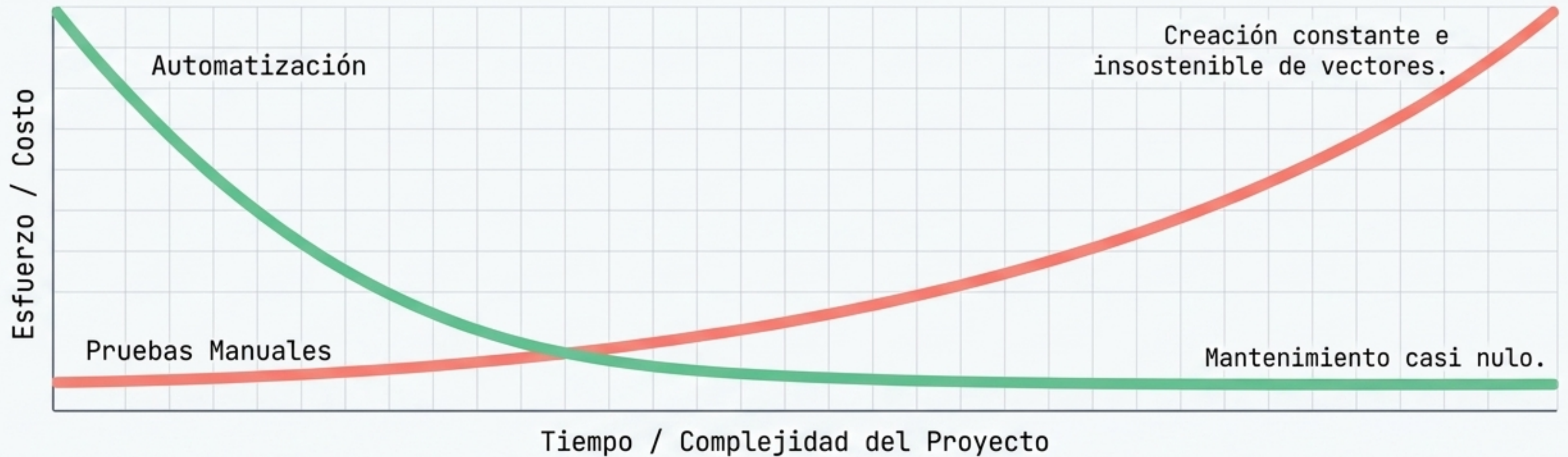
La solución principal para la gestión de pruebas unitarias en entornos Microsoft .NET.

Modo Depuración: Validación de Edades (Caso 1)

Contexto: Un sistema debe validar los años de nacimiento numéricos ingresados por usuarios.
¿Debemos probar todas las fechas de cuatro dígitos? No. Combinar clases de equivalencia con valores límite reduce miles de pruebas potenciales a solo 3 o 4 casos.



El Cuello de Botella Manual vs. Automatización



El Valor de la Automatización: Los frameworks modernos generan casos de prueba para cubrir gran parte del código, detectan errores sistemáticamente y los documentan automáticamente. Vital en entornos de rápida expansión como la computación móvil.

Matriz de Automatización: Código vs. Interfaz

Pruebas de Código (A)	Pruebas de Interfaz Gráfica (B)
Enfoque: Requisitos internos, librerías, módulos y clases.	Enfoque: Interacción externa y flujos reales de usuario.
Complejidad: Se automatizan de manera relativamente sencilla.	Complejidad: Altamente complejas; simulación precisa de hardware.
Interacción: Validación directa con la lógica matemática interna.	Interacción: Simula teclado, eventos de ratón y clics en ventanas.



SELENIUM

Herramienta líder para pruebas de interfaz. Permite grabar, editar, depurar casos de uso y lanzar simulaciones funcionales completas simulando a un usuario real.

Dashboard de Validación: App Compra-Venta (Caso 2)

Contexto: Desplegando una app de analítica de vehículos. Una aplicación pequeña requiere una extensa batería de pruebas.

Vectores de Prueba (4 Total)	
✓	[PASSED] Lógica Financiera: Validación de cálculos de gastos e ingresos (Día, Mes, Año).
✓	[PASSED] Agregación de Datos: Sumatorias anuales vs resúmenes mensuales.
!	[PENDING] Edge Cases de Calendario: Comportamiento en meses de 28 vs 31 días.
!	[PENDING] Excepciones Temporales: Cálculos específicos en años bisiestos.

Terminal

> La solución sistemática: Configurar un sistema que descargue el código, lo compile y lance estos vectores de prueba de forma automatizada cada noche.

El Embudo de la Calidad y la Normativa ISO



[!] **ADVERTENCIA:** Saltar los procedimientos de calidad internos definidos por la empresa resulta en el incumplimiento de la norma y la posible pérdida del certificado en futuras auditorías.

Síntesis: El Pipeline de la Integración Continua



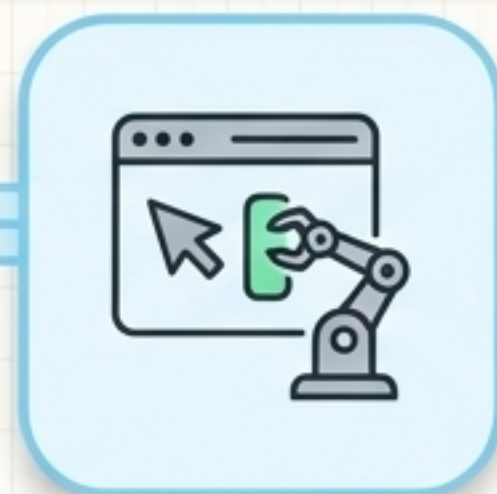
1. Desarrollo y Filtrado

Tratamiento de excepciones + Filtros de cobertura y valores límite.



2. Pruebas Unitarias Aisladas

JUnit, PHPUnit, NUnit validan funciones individuales rápidamente.



3. Automatización Funcional

Selenium ejecuta vectores complejos simulando al usuario real.



4. Auditoría y Despliegue

Cumplimiento estricto del procedimiento para mantener la norma ISO.

La calidad del software no es una fase final de revisión; es un marco de trabajo automatizado, auditable y continuo que se integra en cada línea de código.